Decomposing Multiplication: A Vertical Packing Approach for Faster TFHE

Rostin Shokri and Nektarios Georgios Tsoutsos

University of Delaware
{rostinsh, tsoutsos}@udel.edu

Abstract. Fully Homomorphic Encryption (FHE) enables private data processing on untrusted servers. However, FHE performance remains a critical bottleneck for applications like machine learning, which heavily rely on both non-linear operations (e.g., comparisons) and numerous ciphertext-ciphertext (CxC) and ciphertext-plaintext (CxP) multiplications. While modern FHE schemes like TFHE efficiently handle non-linear operations, their multiplication time complexity remains a significant performance limitation. This paper introduces novel algorithms for CxC and CxP multiplication, as well as the dot-product, a critical kernel in machine learning inference (e.g., convolution). Our method leverages Vertical Packing, decomposing multiplication into a series of efficient, precision-dependent lookup table operations. We evaluate our algorithms and their parallelized variants against the default implementation in TFHE-rs and a recent state-of-the-art work. Our results demonstrate several times faster execution time, significantly accelerating FHE for practical applications.

Keywords: Circuit Bootstrapping · Vertical Packing · Fully Homomorphic Multiplication · TFHE.

1 Introduction

Data-intensive applications, from large-scale genomic processing [5, 18, 19] to advanced machine learning workloads [15, 17, 24], are increasingly migrated to the cloud. This shift to remote data processing creates a paramount concern for data confidentiality [29], as it exposes sensitive information to new risks, such as untrusted servers reading and misusing personal data, as well as side-channel attacks that can extract private data from co-located tenants [31, 32]. While standard encryption like AES [11] protects data at rest and in transit, it is insufficient for end-to-end private computation, as data must be decrypted server-side before it can be processed, creating a window of vulnerability.

An effective solution is Fully Homomorphic Encryption (FHE) [16], which enables direct computation on encrypted data, ensuring that secret keys are never exposed, and no information can be gathered about the underlying plaintext. There are several FHE schemes [3, 7, 14] to choose from that come with different features and benefits. The TFHE (CGGI) scheme [9] in particular is effective for

handling non-linear operations, which are critical for applications like machine learning [21, 25]. This capability is powered by its programmable bootstrapping (PBS) primitive, which can evaluate arbitrary univariate functions on small, encrypted integers.

Beyond academic motivation, TFHE is already being exposed to application developers through Zama's software stack, `tfhe-rs`, which is a pure Rust implementation of TFHE for Boolean and integer arithmetic over encrypted data. It has Rust, C, and WASM APIs as well as hardware-accelerated backends, and its integer API specifically targets values smaller than 256 bits. Moreover, Zama's confidential smart-contract stack uses TFHE-rs.

A simple example is a confidential auction or token transfer. In a confidential auction, the contract must compare an encrypted bid against the encrypted current maximum and conditionally update the winning state without revealing which branch was taken. More generally, confidential smart contracts require exact integer semantics. For instance, a confidential token transfer may need to compute an exact fee, verify that a hidden balance is satisfied by a lower bound, and then update encrypted balances accordingly. These patterns require encrypted multiplication, comparison, and conditional selection on exact integers, which makes TFHE a perfect match. Consequently, improving the performance of TFHE-native integer kernels such as multiplication is not only of theoretical interest, but also directly relevant to emerging applications that operate on encrypted state and exact integer logic.

Despite TFHE's strength in non-linear operations, its performance for arithmetic operations, specifically ciphertext-ciphertext (CxC) as well as ciphertext-plaintext (CxP) multiplication, remains a critical bottleneck. These operations are the dominant cost in many privacy-preserving workloads, including the convolution and linear layers of neural networks. The original PBS paradigm, which operates on small bit-widths, is inefficient for the large-bitwidth multiplications (e.g., 16-bit or 32-bit) required by these workloads.

Recent refinements to the TFHE scheme [26, 28] have introduced a practical and efficient circuit bootstrapping (CBS) operation, which enables the computation of larger bit-width lookup tables (LUT) via the vertical packing (VP) operation. The flexibility of the VP operation and the efficiency of the new CBS bootstrapping make it possible to redesign core algorithms such as AES Transciphering [27]. The primary bottleneck for this approach, the CBS operation, was historically too slow for practical use, having up to $10\times$ overhead compared to a regular PBS [8]. However, recent optimizations have reduced its cost to be competitive with a standard PBS, making this computational model viable.

Although hybrid FHE pipelines that use CKKS for SIMD-efficient linear layers and TFHE for non-linear operations are an important direction, our focus here is on a different setting. Hybrid designs optimize throughput for large packed workloads, but they also depend on approximate arithmetic, scheme switching, and heterogeneous ciphertext representations. In contrast, this work targets pure-TFHE exact integer arithmetic, where staying entirely within TFHE is

desirable for exact 16-bit/32-bit computation, comparison-heavy control flow, which avoids costly scheme-switching overheads.

To date, these new primitives have not been leveraged to create optimized, high-performance arithmetic algorithms. The default multiplication methods in FHE libraries, such as `tfhe-rs`, still rely on an older, PBS-only computation mode, which requires many PBS operations to enable computation on wider bit sizes. For the first time, our work demonstrates a practical performance shift: *arithmetic algorithms utilizing modern CBS and Vertical Packing are now faster than traditional PBS-only approaches*. By leveraging the highly optimized CBS algorithm introduced in [26], we show that the historical assumption that CBS is "too slow" for arithmetic is no longer valid.

In this work, we propose a new set of multiplication algorithms that leverage the VP paradigm to decompose multiplication into a series of efficient, precision-dependent β-bit lookup table operations. Our novel methodology utilizes CBS to refresh ciphertexts, ensuring the composability of our new operations. Compared to recent approaches such as [22], which only support ciphertext-plaintext multiplication, our solution and algorithms provide efficient solutions to both ciphertext-plaintext and ciphertext-ciphertext multiplication. Our primary contributions are as follows:

- We introduce novel algorithms for ciphertext-plaintext (CxP) and ciphertext-ciphertext (CxC) multiplication, which are decomposed into β-bit LUTs and evaluated using vertical packing.
- We propose an efficient ciphertext-plaintext dot-product algorithm, a critical kernel in machine learning applications.
- We optimize the efficiency of our algorithms by utilizing a batched CBS operation, significantly reducing the number of costly PBS operations.
- We analyze new parallelization strategies for these algorithms to maximize throughput on multi-core architectures.

Roadmap: The rest of the paper is organized as follows: In Section 2, we present essential background information on FHE and the TFHE primitives. Section 3 introduces our novel multiplication algorithms, along with parallelization and optimization strategies. In Section 4, we present our experimental evaluation, reporting significant speedups against baseline and state-of-the-art works. Finally, Section 5 discusses notable prior works, and Section 6 presents our concluding remarks.

2 Background

2.1 Fully Homomorphic Encryption (FHE)

Fully Homomorphic Encryption (FHE) provides the capability to perform arbitrary computations directly on encrypted data, thus avoiding the need to decrypt sensitive data during processing. Modern FHE schemes inject noise into ciphertexts, which accumulates with each homomorphic operation. If this noise exceeds

a certain threshold, decryption will fail. To manage this, schemes utilize *bootstrapping*, a process that homomorphically evaluates the decryption function to reset the noise and permit further computation. The security of most contemporary designs is based on problems like Learning with Errors (LWE) [23] and its ring variant (RLWE) [20], which are believed to be post-quantum secure. These schemes fundamentally support homomorphic addition and multiplication, which are sufficient to express any arbitrary computation.

A major category of FHE schemes is designed for *integer* (or word-wise) arithmetic. Schemes such as BFV [14] and BGV [3] can "batch" multiple integers into a single RLWE ciphertext, supporting both addition and multiplication on the packed data. The CKKS scheme [7] provides analogous batching capabilities but is optimized for approximate real-number arithmetic. In contrast, bit-wise schemes like FHEW [13] operate at the level of individual bits; FHEW encrypts each bit into a separate LWE ciphertext, facilitating low-latency evaluation of logic gates. It employs *gate bootstrapping*, a technique that performs a Boolean operation (e.g., `XOR`, `AND`) and resets the ciphertext noise simultaneously.

TFHE [9] extends this bit-wise approach by introducing *programmable bootstrapping* (PBS). At the core of PBS is a fast blind rotation primitive, which, combined with binary secret keys, allows TFHE to evaluate any arbitrary univariate function as part of the noise-refreshing step. A single PBS operation typically completes in tens of milliseconds on a CPU. This mechanism also allows TFHE to efficiently handle small integers (e.g., 1 to 4 bits) using much smaller lattice parameters than those required by word-wise schemes.

2.2 TFHE Primitives

The TFHE scheme's original computing paradigm [9] uses three fundamental operations: PBS, key switching, and modulus switching. Subsequent research [2] introduced a new computing paradigm, by also utilizing circuit bootstrapping and vertical packing, which have enhanced the scheme's performance and expanded its versatility for a wider range of applications. These core operations are described next.

Programmable Bootstrapping (PBS) The PBS operation in TFHE is a key procedure that simultaneously refreshes a noisy LWE ciphertext and computes a user-defined function f over its encrypted message μ_{in}. The high-level process, detailed in [9, Alg 9], consists of three stages. First, `ModSwitch` is applied to the input LWE ciphertext, mapping its modulus down from q to $2N$, where N is the GLWE ring dimension. Next, `BlindRotate` is performed on the modulus-switched ciphertext; this step uses a GLWE accumulator that encodes the lookup table for the function f. Finally, `SampleExtract` retrieves the first polynomial coefficient of the rotated GLWE, resulting in a new LWE ciphertext (under the GLWE secret key S) that encrypts the function's output, $f(\mu_{in})$. The `BlindRotate` step itself is implemented using the homomorphic multiplexer

(CMUX) primitive that relies on the external product, which was introduced in [9].

Circuit Bootstrapping (CBS) In the CBS paradigm, input LWE ciphertexts are lifted into the GGSW format, holding a single bit of message. A GGSW ciphertext is defined by a decomposition depth ℓ and base β, comprising $(k+1)\ell$ GLWE ciphertexts under the secret key $S = (S_0, \ldots, S_k)$. Assuming a single message bit m, these components take the form $\texttt{GLWE}_S(m \cdot S_i \cdot \frac{q}{\beta^j})$ for $1 \leq j \leq \ell$ and $0 \leq i \leq k$. Recent works [26, 28] have introduced new pipelines to significantly reduce CBS latency by splitting the process into two phases: refresh and conversion. First, the **PBSmanyLUT** refresh stage uses a single blind rotation, by omitting sample extraction to generate multiple scaled LWE encryptions of the same bit. Second, a conversion stage assembles these LWE ciphertexts into the final GGSW. Their conversion stage improves upon previous work and involves a **Homomorphic Trace (HomTrace)**, which applies GLWE key switching to isolate the message in monomial terms while zeroing out others, followed by **Scheme Switching (SS)**, which uses an external product to combine the ciphertexts and secret key into a valid GGSW. For a detailed algorithmic overview, we refer the reader to [26, 28].

2.3 Vertical Packing (VP)

In intuitive terms, Vertical Packing (VP) can be viewed as a way to evaluate a wider look-up table by using several encrypted bits together as the table index. Rather than bootstrapping each small operation separately, VP packs the selection logic into one procedure that outputs the LUT value corresponding to the encrypted input. In this sense, VP acts like a homomorphic table lookup for larger integers: the encrypted input chooses a row of the table, and the server cannot infer which row was selected. Evaluating a Look-Up Table (LUT) for m GGSW-encrypted input bits via VP incurs a cost that directly depends on the number of inputs m relative to the GLWE ring dimension N. The process is highly efficient when the input size is small ($m \leq \log_2 N$). In this case, the evaluation only requires a single blind rotation (BR), which costs just $\log_2 N$ CMUX operations. This is a major advantage over a standard PBS, which would require n CMUX operations for its BR (where $n \gg \log_2 N$ in typical parameters). However, if the number of inputs is large ($m > \log_2 N$), the cost increases significantly. An initial CMUX tree is required to reduce the inputs *before* the BR. This tree adds a cost of $2^{m-\log_2 N} - 1$ CMUX operations, which is exponential in the number of "excess" inputs. On the output side, a single VP evaluation produces an LWE ciphertext capable of encoding $\log_2 p$ bits, where p is the message modulus. This capacity limit means a full m-to-m bit LUT must be partitioned into $\frac{m}{\log_2 p}$ separate VP evaluations. For example, an 8-to-8 bit LUT using a modulus of $p = 2$ (where $\log_2 2 = 1$ bit) would require 8 distinct VP runs to compute the full 8-bit output.

2.4 Threat Model

We consider the standard outsourced-computation setting for FHE: a client encrypts inputs under their secret key and sends the encrypted ciphertexts and public evaluation keys to an untrusted server for homomorphic evaluation. The adversary is honest-but-curious and may observe all ciphertexts, public keys, and intermediate encrypted states, but does not know the secret key. While standard TFHE provides `IND-CPA` security, our goal is targeting a stronger threat model, $\texttt{IND-CPA}^D$. This threat model was proposed to defend against key recovery attacks proposed in [6], which uses a decryption oracle to attack the approximate nature of decryption. While the main operation of TFHE, the programmable bootstrap (PBS), produces exact results, it introduces a small failure probability, P_{fail}, due to the high noise growth during modulus switching. Therefore, we target a correctness failure probability of $P_{\text{fail}} = 2^{-128}$, to ensure security against the attack, since the P_{fail} becomes a security parameter in this threat model. We generate and use parameters that achieve this probability of failure for both our work and related works. We do not address malicious-server verifiability, access-pattern leakage, or implementation-level side channels such as timing, cache, or cross-tenant attacks.

3 Our Proposed Approach

In this section, we formalize the framework underlying our vertical packing–based multiplication algorithms in TFHE. We first present ciphertext–plaintext (CxP) multiplication, where an n-bit input is decomposed into β-bit limbs, lifted from LWE to GGSW via circuit bootstrapping, and processed through vertical packing LUTs to produce base 2^β partial products. We then reuse the same techniques to construct ciphertext–ciphertext (CxC) multiplication and derive ciphertext-plaintext dot product (CxP-DP). We enhance performance by using batched CBS, reducing CBS and VP computations, and describe a parallelization strategy that exploits the independence of CBS and VP calls to obtain scalable performance on multi-core CPUs.

3.1 Ciphertext-Plaintext Multiplication

To explain the methodology of our multiplication algorithms, we first show how we decompose an n-bit multiplication into a sequence of base-2^β limb products and carry-propagation steps that can be realized using CBS, VP, and β-bit ciphertext-ciphertext addition subroutine.

Choosing optimal β For all our algorithms, we choose $\beta = 16$, since this is the optimal number for which a 16-bit VP LUT remains efficient, and reduces the number of intermediate CBS operations. In particular, the ring dimension N we choose for our TFHE parameters is either $N = 1024$ or $N = 2048$. Therefore, the VP operation consists of $\log_2(N) + 2^{(16-\log_2(N))} - 1$ CMUX operations (which

equals to either 73 or 42 CMUX operations respectively). A PBS requires around an order of magnitude more CMUX operations, making it significantly more expensive. Choosing $\beta = 8$ can effectively reduce the VP execution time (to just 8 CMUX operations), but this saving is negated by the increased number of intermediate CBS operations required due to the rise in limb products. Since VP already has a much lower relative latency than PBS (a sub-procedure used in CBS), the overall performance gain is minimal.

Our Methodology We assume n is a multiple of 16, the bit width of the LUT we decompose to. The underlying LWE scheme has plaintext modulus $p = 2^\kappa$, so each LWE ciphertext encrypts a value in $\mathbb{Z}_p$ (for $\kappa = 1$, this is a single bit).

We represent each message in $\mathbb{Z}_{2^n}$ as LWE ciphertexts encrypting chunks of message in $\mathbb{Z}_{2^\kappa}$, arranged into t blocks holding 16 bits of message, divided into $\frac{16}{\kappa}$ positions:

$$t := \frac{n}{16}, \quad \mathbf{Z}_i = \left(\mathsf{Enc}_{\mathrm{LWE}}(z_{i,0}), \ldots, \mathsf{Enc}_{\mathrm{LWE}}(z_{i,\frac{16}{\kappa}-1})\right), \quad 0 \le i \le t-1.$$

The first step is to apply CBS (or batched CBS if $\kappa \ge 2$) to each LWE ciphertext. In normal CBS, where $\kappa = 1$, it converts 16 LWE ciphertexts to 16 GGSW ciphertexts. In the batched CBS case, assume we have $\kappa = 2$, then we have 8 LWE ciphertexts, each holding *two* bits; so, the batched CBS converts each LWE holding two bits into two GGSW ciphertexts, each holding one of the bits. Therefore we get:

$$\mathbf{X}_i := \left(\mathsf{Enc}_{\mathrm{GGSW}}(z_{i,0}), \ldots, \mathsf{Enc}_{\mathrm{GGSW}}(z_{i,15})\right)$$

for the GGSW-controlled inputs to vertical packing LUTs.

We view X as decomposed into t base-B limbs,

$$X = \sum_{i=0}^{t-1} X_i B^i, \qquad X_i \in \{0, \ldots, B-1\}. \tag{1}$$

Scalar decomposition. Let $S \in \mathbb{Z}_{2^n}$ be the plaintext scalar. We also decompose it into base B:

$$S = \sum_{j=0}^{t-1} S_j B^j, \qquad S_j \in \{0, \ldots, B-1\}. \tag{2}$$

Product in base $B = 2^{16}$. Using (1) and (2), the product expands:

$$SX = \left(\sum_{j=0}^{t-1} S_j B^j\right)\left(\sum_{i=0}^{t-1} X_i B^i\right) = \sum_{i=0}^{t-1}\sum_{j=0}^{t-1} P_{i,j} B^{i+j}, \tag{3}$$

where $P_{i,j} := S_j X_i$ are 32-bit partial products with $0 \le P_{i,j} < B^2 = 2^{32}$. We write each partial product as

$$P_{i,j} = P_{i,j}^{(0)} + B\, P_{i,j}^{(1)}, \tag{4}$$

with low and high 16-bit parts $P_{i,j}^{(0)} \coloneqq P_{i,j} \bmod B$, $P_{i,j}^{(1)} \coloneqq \left\lfloor \frac{P_{i,j}}{B} \right\rfloor$. Substituting (4) into (3) gives

$$SX = \sum_{i,j} \left(P_{i,j}^{(0)} + BP_{i,j}^{(1)}\right) B^{i+j} = \sum_{i,j} P_{i,j}^{(0)} B^{i+j} + \sum_{i,j} P_{i,j}^{(1)} B^{i+j+1}. \tag{5}$$

Modulo $B^t = 2^n$, only exponents $\leq t-1$ remain. For each limb $k = 0, \ldots, t-1$, the (unnormalized) coefficient is

$$Z_k \coloneqq \sum_{i+j=k} P_{i,j}^{(0)} + \sum_{i+j+1=k} P_{i,j}^{(1)}. \tag{6}$$

We therefore have

$$SX \equiv \sum_{k=0}^{t-1} Z_k B^k \pmod{B^t}.$$

To obtain proper base-B limbs we normalize by adding the carries from previous computations. Assume $C_0 = 0$, we have:

$$Y_k \coloneqq (Z_k + C_k) \bmod B, \qquad 0 \leq k \leq t-1, \tag{7}$$

$$C_{k+1} \coloneqq \left\lfloor \frac{Z_k + C_k}{B} \right\rfloor, \qquad 0 \leq k \leq t-1, \tag{8}$$

so that $Y_k \in \{0, \ldots, B-1\}$ and $SX \equiv \sum_{k=0}^{t-1} Y_k B^k \pmod{B^t}$.

Realization via VP and 16-bit CxC addition For each scalar limb S_j we define two GLWE LUTs $f_j^{(0)}(u) = (S_j u) \bmod B$, and $f_j^{(1)}(u) = \lfloor \frac{S_j u}{B} \rfloor$, with $u \in \{0, \ldots, B-1\}$. We apply two sets of 16-bit VP operations $\mathbf{P}_{i,j}^{(0)} \leftarrow \mathsf{VP}_{16}^{(p)}\left(\mathbf{X}_i, f_j^{(0)}\right)$ and $\mathbf{P}_{i,j}^{(1)} \leftarrow \mathsf{VP}_{16}^{(p)}\left(\mathbf{X}_i, f_j^{(1)}\right)$ to the GGSW vector $\mathbf{X}_i$, outputting $\frac{16}{\kappa}$ LWE ciphertexts over $\mathbb{Z}_p$.

Now, when adding the partial products for each limb index k, we maintain an LWE state $\mathbf{Y}_k$ ($\frac{16}{\kappa}$ LWE ciphertexts) Whenever a new low part $P_{i,j}^{(0)}$ or high part $P_{i,j}^{(1)}$ contributes to limb k, we update with $(\mathbf{Y}_k, \mathbf{C}) \leftarrow \mathsf{Add16CXC}^{(p)}(\mathbf{Y}_k, \mathbf{P})$, where $\mathsf{Add16CXC}^{(p)}$ is a 16-bit ciphertext-ciphertext addition subroutine over $\mathbb{Z}_p$ LWE ciphertexts, and $\mathbf{C}$ is a carry block to be propagated to $\mathbf{Y}_{k+1}$, consistent with the recursion on C_k.

If at some later stage a $\mathbf{Y}_k$ must be used as VP input, each LWE slot in $\mathbf{Y}_k$ is first circuit-bootstrapped back to GGSW, and the resulting GGSW vector is fed into another VP. In this way we obtain all limbs $Y_0, \ldots, Y_{t-1}$ as 16-slot LWE blocks over $\mathbb{Z}_p$, realizing the exact product $Y \equiv SX \pmod{2^n}$ using only CBS, VP, and 16-bit ciphertext–ciphertext additions. We present our algorithm for CxP multiplication in Algorithm 1.

Algorithm 1 Homomorphic n-bit scalar–ciphertext multiplication via CBS, VP, and 16-bit CxC addition

Input: n multiple of 16; $B = 2^{16}$; $t \leftarrow n/16$; Plaintext scalar $S \in \{0, \ldots, 2^n - 1\}$.
Input: Plaintext modulus $p = 2^\kappa$ (each LWE holds κ bits).
Input: LWE encryptions encoding the input integer X as $\mathbf{Z}_i = (\mathsf{Enc}_{\mathrm{LWE}}(z_{i,0}), \ldots, \mathsf{Enc}_{\mathrm{LWE}}(z_{i,\frac{16}{\kappa}-1}))$, for $i = 0, \ldots, t-1$.
Output: For each $k = 0, \ldots, t-1$, a 16-bit output block $\mathbf{Y}_k = (\mathsf{Enc}_{\mathrm{LWE}}(y_{k,0}), \ldots, \mathsf{Enc}_{\mathrm{LWE}}(y_{k,\frac{16}{\kappa}-1}))$ over $\mathbb{Z}_p$, encoding the k-th limb of $Y \equiv SX \pmod{2^n}$.

$S = \sum_{j=0}^{t-1} S_j B^j$, with $S_j \in \{0, \ldots, B-1\}$.
for $i = 0$ to $t-1$ **do**
  for $r = 0$ to $\frac{16}{\kappa} - 1$ **do**
    $(\mathsf{Enc}_{\mathrm{GGSW}}(z_{i,\kappa r}), \ldots, \mathsf{Enc}_{\mathrm{GGSW}}(z_{i,\kappa r+\kappa-1})) \leftarrow \mathsf{CBS}^{(\kappa)}(\mathsf{Enc}_{\mathrm{LWE}}(z_{i,r}))$ ▷ *regular CBS if $\kappa = 1$, batched CBS if $\kappa > 1$*
  $\mathbf{X}_i \leftarrow (\mathsf{Enc}_{\mathrm{GGSW}}(z_{i,0}), \ldots, \mathsf{Enc}_{\mathrm{GGSW}}(z_{i,15}))$
for $k = 0$ to $t-1$ **do**
  $\mathbf{Y}_k \leftarrow \mathbf{0}^{16/\kappa}$ ▷ *all-zero 16-bit block over $\mathbb{Z}_p$*
  View $\mathbf{Y}_k = (\mathbf{Y}_{k,\mathrm{lo}}, \mathbf{Y}_{k,\mathrm{hi}})$, where each half is an 8-bit block of size $8/\kappa$
$\mathbf{0}_{\mathrm{lo}}, \mathbf{0}_{\mathrm{hi}} \leftarrow$ encrypted 8-bit zero blocks
$C_{\mathrm{zero}} \leftarrow \mathsf{Enc}_{\mathrm{LWE}}(0)$ ▷ *encrypted zero carry bit*
for $i = 0$ to $t-1$ **do**
  for $r = 0$ to $\frac{16}{\kappa} - 1$ **do**
    $(\mathsf{Enc}_{\mathrm{GGSW}}(z_{i,\kappa r}), \ldots, \mathsf{Enc}_{\mathrm{GGSW}}(z_{i,\kappa r+\kappa-1})) \leftarrow \mathsf{CBS}^{(\kappa)}(\mathsf{Enc}_{\mathrm{LWE}}(z_{i,r}))$ ▷ *regular CBS if $\kappa = 1$, batched CBS if $\kappa > 1$*
  $\mathbf{X}_i \leftarrow (\mathsf{Enc}_{\mathrm{GGSW}}(z_{i,0}), \ldots, \mathsf{Enc}_{\mathrm{GGSW}}(z_{i,15}))$
for $k = 0$ to $t-1$ **do**
  $\mathbf{Y}_k \leftarrow \mathbf{0}^{16/\kappa}$ ▷ *all-zero 16-bit block over $\mathbb{Z}_p$*
  View $\mathbf{Y}_k = (\mathbf{Y}_{k,\mathrm{lo}}, \mathbf{Y}_{k,\mathrm{hi}})$, where each half is an 8-bit block of size $8/\kappa$
$\mathbf{0}_{\mathrm{lo}}, \mathbf{0}_{\mathrm{hi}} \leftarrow$ encrypted 8-bit zero blocks
$C_{\mathrm{zero}} \leftarrow \mathsf{Enc}_{\mathrm{LWE}}(0)$ ▷ *encrypted zero carry bit*
for $j = 0$ to $t-1$ **do**
  Define GLWE LUTs

$$f_j^{(0)}(u) = (S_j u) \bmod B, \qquad f_j^{(1)}(u) = \left\lfloor \frac{S_j u}{B} \right\rfloor,$$

  where $u \in \{0, \ldots, B-1\}$.
  for $i = 0$ to $t-1$ **do**
    $k \leftarrow i + j$
    $\mathbf{P}_{i,j}^{(0)} \leftarrow \mathsf{VP}_{16}^{(p)}\big(\mathbf{X}_i, f_j^{(0)}\big)$ ▷ *$16/\kappa$ LWEs over $\mathbb{Z}_p$*
    $\mathbf{P}_{i,j}^{(1)} \leftarrow \mathsf{VP}_{16}^{(p)}\big(\mathbf{X}_i, f_j^{(1)}\big)$ ▷ *$16/\kappa$ LWEs over $\mathbb{Z}_p$*
    View $\mathbf{P}_{i,j}^{(0)} = (\mathbf{P}_{i,j,\mathrm{lo}}^{(0)}, \mathbf{P}_{i,j,\mathrm{hi}}^{(0)})$ and $\mathbf{P}_{i,j}^{(1)} = (\mathbf{P}_{i,j,\mathrm{lo}}^{(1)}, \mathbf{P}_{i,j,\mathrm{hi}}^{(1)})$, where each half is an 8-bit block of size $8/\kappa$.
    if $k < t$ **then**
      $\big((\mathbf{Y}_{k,\mathrm{lo}}, \mathbf{Y}_{k,\mathrm{hi}}), C\big) \leftarrow \mathsf{Add16CXC}^{(p)}\big(\mathbf{Y}_{k,\mathrm{lo}}, \mathbf{Y}_{k,\mathrm{hi}}, \mathbf{P}_{i,j,\mathrm{lo}}^{(0)}, \mathbf{P}_{i,j,\mathrm{hi}}^{(0)}, C_{\mathrm{zero}}\big)$
      if $k + 1 < t$ **then**
        $\big((\mathbf{Y}_{k+1,\mathrm{lo}}, \mathbf{Y}_{k+1,\mathrm{hi}}), C\big) \leftarrow \mathsf{Add16CXC}^{(p)}\big(\mathbf{Y}_{k+1,\mathrm{lo}}, \mathbf{Y}_{k+1,\mathrm{hi}}, \mathbf{P}_{i,j,\mathrm{lo}}^{(1)}, \mathbf{P}_{i,j,\mathrm{hi}}^{(1)}, C\big)$
        for $r = k + 2$ to $t - 1$ **do**
          $\big((\mathbf{Y}_{r,\mathrm{lo}}, \mathbf{Y}_{r,\mathrm{hi}}), C\big) \leftarrow \mathsf{Add16CXC}^{(p)}\big(\mathbf{Y}_{r,\mathrm{lo}}, \mathbf{Y}_{r,\mathrm{hi}}, \mathbf{0}_{\mathrm{lo}}, \mathbf{0}_{\mathrm{hi}}, C\big)$
return $\mathbf{Y}_0, \ldots, \mathbf{Y}_{t-1}$

16-bit Ciphertext-Ciphertext Addition An n-bit ciphertext addition using `tfhe-rs` is implemented by utilizing a carry space for each LWE ciphertext, where intermediate additions are stored in the LWE ciphertexts, and is propagated using a PBS. Since the carry space must be empty for subsequent operations, another PBS is used to empty the carry space. In our algorithms, we do not support any carry space; therefore, we must compute the additions with the VP LUTs. One VP LUT is used to add the ciphertext bits, represented with the function f_{sum}, and we use another VP LUT to get the carry and propagate to the next limb computation, with the function f_{carry}. The algorithm for 16-bit ciphertext addition follows the blueprint of CxP given in Algorithm 2. The algorithm inputs two LWE encrypted operands X and Y, and a carry bit C_{in}.

Algorithm 2 16-bit ciphertext–ciphertext addition $\mathsf{Add16CXC}^{(p)}$

Input: $X_{\text{lo}}, X_{\text{hi}}, Y_{\text{lo}}, Y_{\text{hi}}$ (8-bit LWE blocks over $\mathbb{Z}_p$), C_{in} (1-bit LWE over $\mathbb{Z}_p$)
Output: $Z_{\text{lo}}, Z_{\text{hi}}$ (8-bit LWE blocks over $\mathbb{Z}_p$), C_{out} (1-bit LWE over $\mathbb{Z}_p$)

1: $(X^G_{\text{lo}}, X^G_{\text{hi}}, Y^G_{\text{lo}}, Y^G_{\text{hi}}, C^G_{\text{in}}) \leftarrow \mathsf{CBS}(X_{\text{lo}}, X_{\text{hi}}, Y_{\text{lo}}, Y_{\text{hi}}, C_{\text{in}})$
2: $Z_{\text{lo}} \leftarrow \mathsf{VP}^{(p)}_{17}\big(X^G_{\text{lo}}, Y^G_{\text{lo}}, C^G_{\text{in}}, f_{\text{sum},0}\big)$
3: $C_1 \leftarrow \mathsf{VP}^{(p)}_{17}\big(X^G_{\text{lo}}, Y^G_{\text{lo}}, C^G_{\text{in}}, f_{\text{carry},0}\big)$
4: $C^G_1 \leftarrow \mathsf{CBS}(C_1)$
5: $Z_{\text{hi}} \leftarrow \mathsf{VP}^{(p)}_{17}\big(X^G_{\text{hi}}, Y^G_{\text{hi}}, C^G_1, f_{\text{sum},1}\big)$
6: $C_{\text{out}} \leftarrow \mathsf{VP}^{(p)}_{17}\big(X^G_{\text{hi}}, Y^G_{\text{hi}}, C^G_1, f_{\text{carry},1}\big)$
7: **return** $(Z_{\text{lo}}, Z_{\text{hi}}), C_{\text{out}}$

Small running example. For intuition, consider a toy radix-B multiplication with two limbs:

$$X = X_0 + BX_1, \qquad S = S_0 + BS_1.$$

Then

$$SX = X_0S_0 + B(X_0S_1 + X_1S_0) + B^2X_1S_1.$$

Modulo B^2, the last term vanishes, so the output consists of a low limb from X_0S_0 and a high limb obtained by accumulating the carry from X_0S_0 together with the cross terms X_0S_1 and X_1S_0. Our homomorphic algorithm follows exactly this structure: CBS lifts encrypted limb blocks to GGSW, VP computes the low/high partial products, and `Add16CXC` merges them.

Batched CBS Optimization To improve the efficiency of our algorithm, we optimize it by employing a batched CBS [26], which can compute the scaled variants of multiple bits τ stored in a single LWE ciphertext with a single blind rotation. It utilizes the `PBSmanyLUT` [10] and multi-value bootstrapping [4]. Doing so requires larger cryptographic parameters, which we account for in our parameter sets. The higher the value τ is, the larger the noise growth. The output of the `PBSmanyLUT` is the scaled LWE ciphertexts of the τ bits. For the conversion

step, however, the HomTrace operation and the scheme switching (SS) operation are still done on each bit separately. Since the blind rotation in the `PBSmanyLUT` is the most costly operation by far, this significantly increases the efficiency of our algorithms. For example, in Algorithm 1, we set $\kappa = \tau$, and a batched CBS is computed instead of a regular CBS.

Note that the number of VP operations also decreases, as the output LWE should also hold $p = 2^\tau$ bits, to allow batched CBS for subsequent computations.

Design tradeoffs. The main computational tradeoff in our design is between the cost of evaluating a wider VP LUT and the number of intermediate CBS and addition stages induced by a smaller radix. Choosing $\beta = 16$ reduces the number of limb products and intermediate refreshes, while keeping VP within the efficient regime of our parameter set. Therefore, our method replaces many small PBS-style digit operations with a smaller number of wider CBS+VP operations.

A second consideration is composability. We introduce CBS between arithmetic stages not only to convert LWE ciphertexts into the GGSW form required by VP, but also to refresh noise through its underlying PBS step, thereby enabling further multi-stage computation. This means that CBS is both a format-conversion mechanism and a composability mechanism in our framework.

Finally, this design comes with an encoding-overhead tradeoff. Lifting compact LWE limb blocks into GGSW increases the size of intermediate ciphertexts, but it allows each refreshed block to drive wider LUT evaluations and reduces the number of repeated small-digit PBS computations. Therefore, the proposed method is best viewed as optimizing the latency/parallelism tradeoff of pure-TFHE arithmetic, rather than minimizing intermediate ciphertext footprint.

3.2 Ciphertext-Plaintext Dot Product

Given a plaintext vector $S = (S^{(0)}, \ldots, S^{(m-1)})$ and an encrypted vector $X = (X^{(0)}, \ldots, X^{(m-1)})$ of n-bit integers in $\mathbb{Z}_{2^n}$, the ciphertext–plaintext dot product is

$$D \coloneqq \sum_{\ell=0}^{m-1} S^{(\ell)} X^{(\ell)} \bmod 2^n.$$

We compute it by reusing our CxP multiplication as a black box: for each coordinate ℓ we homomorphically evaluate the product $S^{(\ell)}X^{(\ell)}$ via Algorithm 1, and then sum all results limb-wise using the same 16-bit ciphertext–ciphertext adder and carry propagation as in the CxP case.

3.3 Ciphertext-Ciphertext Multiplication

For ciphertext–ciphertext (CxC) multiplication, we follow the same long-multiplication structure as in the CxP case: both encrypted inputs $X, Y \in \mathbb{Z}_{2^n}$ are decomposed into base-B limbs $X = \sum_i X_i B^i$, $Y = \sum_j Y_j B^j$, we form partial products $P_{i,j} = X_i Y_j$, split each $P_{i,j}$ into low/high 16-bit halves, and accumulate them with 16-bit CxC addition and carry propagation as in Equations (6)–(8). The key difference is how we obtain $P_{i,j}$ homomorphically: since

Algorithm 3 Homomorphic CxP dot product via CxP multiplication and 16-bit CxC addition

Input: Vector length m; bit width n multiple of 16; $B = 2^{16}$; $t \leftarrow n/16$.
Input: Plaintext scalars $S^{(0)}, \ldots, S^{(m-1)} \in \{0, \ldots, 2^n - 1\}$.
Input: For each λ, LWE encryptions of $X^{(\lambda)} \in \mathbb{Z}_{2^n}$ as blocks $\mathbf{Z}_0^{(\lambda)}, \ldots, \mathbf{Z}_{t-1}^{(\lambda)}$.
Output: LWE blocks $\mathbf{D}_0, \ldots, \mathbf{D}_{t-1}$ encoding $D = \sum_{\lambda=0}^{m-1} S^{(\lambda)} X^{(\lambda)} \bmod 2^n$.

for $k = 0$ to $t-1$ **do**
  $\mathbf{D}_k \leftarrow \mathbf{0}^{16/\kappa}$ ▷ *all-zero 16-bit LWE block over* $\mathbb{Z}_p$
  View $\mathbf{D}_k = (\mathbf{D}_{k,\mathrm{lo}}, \mathbf{D}_{k,\mathrm{hi}})$, where each half is an 8-bit block of size $8/\kappa$
for $\lambda = 0$ to $m-1$ **do**
  $(\mathbf{Y}_0^{(\lambda)}, \ldots, \mathbf{Y}_{t-1}^{(\lambda)}) \leftarrow \mathsf{CxP_Mul}\big(S^{(\lambda)}, \mathbf{Z}_0^{(\lambda)}, \ldots, \mathbf{Z}_{t-1}^{(\lambda)}\big)$ ▷ *Call Algorithm 1*
  $C_0 \leftarrow \mathsf{Enc}_{\mathrm{LWE}}(0)$ ▷ *encrypted zero carry bit*
  for $k = 0$ to $t-1$ **do**
    View $\mathbf{Y}_k^{(\lambda)} = (\mathbf{Y}_{k,\mathrm{lo}}^{(\lambda)}, \mathbf{Y}_{k,\mathrm{hi}}^{(\lambda)})$, where each half is an 8-bit block of size $8/\kappa$
    $\big((\mathbf{D}_{k,\mathrm{lo}}, \mathbf{D}_{k,\mathrm{hi}}), C_{k+1}\big) \leftarrow \mathsf{Add16CXC}^{(p)}\big(\mathbf{D}_{k,\mathrm{lo}}, \mathbf{D}_{k,\mathrm{hi}}, \mathbf{Y}_{k,\mathrm{lo}}^{(\lambda)}, \mathbf{Y}_{k,\mathrm{hi}}^{(\lambda)}, C_k\big)$
    ▷ *discard* C_t *since computation is modulo* 2^n
return $\mathbf{D}_0, \ldots, \mathbf{D}_{t-1}$

both operands are encrypted, we now apply CBS to *both* inputs and implement the map $(u, v) \mapsto u \cdot v$ via 16-bit VP LUTs whose inputs are two encrypted chunks of size 8 bits each. In other words, each logical base-B limb is internally represented as two 8-bit chunks per operand so that any cross-multiplied pair (u, v) fits into a 16-bit LUT; VP evaluates these cross-products, which are homomorphically accumulated to form the full $P_{i,j}$. The resulting low and high 16-bit components are fed into the same accumulation and carry pipeline as in CxP multiplication. Thus, CxC multiplication reuses the CxP framework verbatim, but with doubled CBS cost (both inputs are circuit-bootstrapped) and 8-bit chunks per encrypted limb to respect the 16-bit LUT constraint.

Ciphertext–ciphertext dot products can then be derived by combining this CxC multiplication with the dot-product blueprint of Section 3.2, replacing each CxP scalar multiplication by a CxC multiplication and summing the resulting products with the same 16-bit CxC adder.

Implementation and Parallelization The parallelism in our framework follows directly from the data-flow of our Algorithms. The expensive front-end operations CBS and VP, act on independent ciphertext blocks and partial products, and therefore admit coarse-grained task parallelism. In CxP multiplication, each input block can be circuit-bootstrapped independently, and each partial product LUT evaluation can be scheduled independently. The same structure carries over to CxC multiplication, with additional parallelism from circuit bootstrapping both operands, and to CxP-DP, where the per-coordinate multiplications are also independent.

Table 1. Our parameter set for our multiplication algorithms, optimized for $\tau = 2$.

n	N	k	ℓ_{ks}	$\mathcal{B}_{ks}$	ℓ_{pbs}	$\mathcal{B}_{pbs}$	ℓ_{tr}	$\mathcal{B}_{tr}$	ℓ_{ss}	$\mathcal{B}_{ss}$	ℓ_{ggsw}	$\mathcal{B}_{ggsw}$	ϑ	τ	$P_{\mathbf{fail}}$
768	1024	2	4	2^3	2	2^{15}	5	2^8	2	2^{17}	2	2^7	1	2	$2^{-157.25}$

Table 2. Default parameter set from `tfhe-rs`.

n	N	k	ℓ_{pbs}	$\mathcal{B}_{pbs}$	ℓ_{ks}	$\mathcal{B}_{ks}$	$P_{\mathbf{fail}}$
918	2048	1	1	2^{23}	4	2^4	$2^{-129.581}$

4 Experimental Evaluation

We implement our proposed algorithms in `tfhe-rs` [30], a state-of-the-art TFHE library written in Rust. We adopt the refined CBS algorithm from [26], and modify it to be compatible with the latest version of `tfhe-rs`, and use the new Centered Mean Noise Reduction from [12] to reduce modulus switching errors, which allows us to generate more efficient parameters for the same P_{fail}.

We compare the efficiency and performance of our algorithms against the default operations provided by `tfhe-rs`, and the current state-of-the-art work [22] that proposes efficient Plaintext Ciphertext multiplication and Plaintext Ciphertext dot-product. We also compare the parallelized versions of our algorithms with `tfhe-rs` and [22]. We target a failure probability of $P_{\text{fail}} = 2^{-128}$ for all of our experiments. All experiments were conducted on an Ubuntu workstation equipped with an AMD 7970X CPU and 160 GB of RAM. We remark that all parameter sets are configured to provide 128 bits of cryptographic security, based on the lattice estimator [1], which is standard in the literature.

4.1 Our Cryptographic Parameters

In terms of performance, the PBS and HomTrace operations are the most expensive. For memory, the primary overhead comes from GGSW ciphertexts, making it crucial to minimize the ℓ_{ggsw} parameter. Our judiciously optimized parameters are presented in Table 1. We set ℓ_{ggsw} to the minimum possible value, as a value of 1 would lead to uncontrollable noise growth. We employ batched CBS with $\tau = 2$, which achieves a failure probability of $P_{\text{fail}} \approx 2^{-128}$.

For both `tfhe-rs` and [22] experiments, we use the default parameter set that is used in the high level API of the `tfhe-rs` library, and show it in Table 2. This parameter set also uses the Centered Mean Noise Reduction technique to optimize the parameters and noise, and targets the same probability of failure.

4.2 Evaluation Key Size Comparison

Table 3 compares the evaluation-key footprint of the default PBS-only `tfhe-rs` context against our custom CBS+VP context. The default context contains the standard Fourier PBS key and an LWE key switching key, while our CBS+VP

Table 3. Evaluation-key size comparison (MB).

Evaluation key component	PBS	CBS+VP
Fourier PBS bootstrapping key	60.162	113.246
LWE key switching key	60.228	–
Pre-PBS GLWE key switching key	–	0.459
Automorphism keys (total)	–	2.458
Scheme switching key	–	0.295
Total	120.390	116.457

context uses a larger Fourier PBS key together with additional key material for pre-PBS GLWE key switching, automorphisms, and scheme switching.

An important point is that the pre-PBS GLWE key switching key is much smaller than the standard LWE key switching key. This is because its size scales with the GLWE secret dimension k, which is a small constant, whereas the LWE key switching key scales with the LWE secret dimension n, which is much larger. Although a GLWE ciphertext is itself larger than an LWE ciphertext, the number of switched secret components is dramatically smaller. This is why, despite supporting the richer CBS+VP functionality, the total evaluation-key material of our custom context is slightly smaller than the default `tfhe-rs` setting: 116.46 MB versus 120.39 MB.

4.3 CxP and CxC Multiplication Results

We evaluate our ciphertext-plaintext (CxP) and ciphertext-ciphertext (CxC) multiplication algorithms against the baseline implementations in `tfhe-rs` [30] and the work from Pottier et al. [22]. Notably, for CxC, we only compare against `tfhe-rs` [30], as the work from [22] does not support this operation type, since they rely on the redundancy in the LUTs, induced by a plaintext operand that reduces the PBS count. We highlight that our algorithms work effectively regardless of whether there is a plaintext operand or not, which makes our CxC methodology uniquely valuable. For both CxP and CxC, we experiment for various bit sizes and use the parameter set from Table 1 for our scheme and the parameter set from Table 2 for the related works. We also exploit thread-level parallelism and report latencies for 1, and 8 CPU threads. The resulting execution times and speedups are summarized in Tables 4 and 5.

4.4 CxP-DP Operation Results

We evaluate our ciphertext–plaintext dot product (CxP-DP) algorithm against `tfhe-rs` [30] and [22]. We consider vector lengths $m \in \{2, 4, 8, 16\}$ of 16-bit entries, using the same parameter sets from Tables 1 and 2. Our scheme reuses the CxP multiplication as a building block and exploits thread-level parallelism across CBS and VP operations. We report latencies for 1, 8, and 16 CPU threads.

Table 4. Execution time (s) of ciphertext-plaintext (CxP) multiplication for different bit-widths and thread counts. All executions target $P_{\text{fail}}=2^{-128}$. We also report the approximate lifted input GGSW overhead under the parameters of Table 1.

Type	Overhead (MB)	Threads	[30] (s)	[22] (s)	Ours (s)	Speedup (×)
		1	0.14	0.11	**0.08**	(1.75, 1.37)
8-bit	1.180	8	0.05	0.05	**0.04**	(1.2, 1.2)
		1	0.52	0.50	**0.15**	(3.46, 3.33)
16-bit	2.359	8	0.09	0.08	**0.06**	(1.5, 1.33)
		1	1.06	1.02	**0.55**	(1.92, 1.85)
32-bit	4.719	8	0.22	0.21	**0.17**	(1.29, 1.23)

Table 5. Execution time (s) of ciphertext-ciphertext (CxC) multiplication for different bit-widths and thread counts. All executions target $P_{\text{fail}}=2^{-128}$. We also report the approximate lifted input GGSW overhead under the parameters of Table 1.

Type	Overhead (MB)	Threads	TFHE-rs [30] (s)	Ours (s)	Speedup (×)
		1	0.42	**0.14**	3.00
8-bit	2.359	8	0.09	**0.04**	2.25
		1	1.69	**0.59**	2.86
16-bit	4.719	8	0.25	**0.11**	2.27
		1	6.59	**2.80**	2.35
32-bit	9.437	8	0.90	**0.52**	1.73

The resulting execution times and speedups for CxP-DP are summarized in Table 6. As the vector size m grows, the number of additions and carry propagation increases due to the lack of carry space, which raises the latency overhead.

5 Related Works

There has been substantial work focusing on accelerating TFHE-based integer arithmetic at the library level, most prominently in `tfhe-rs` [30]. The high-level integer API represents an n-bit encrypted integer as a radix decomposition with the default parameter set, where each digit stores 2 message bits and 2 carry bits and 1 padding bit, and an n-bit value is encoded as $k = \lceil \ell/2 \rceil$ LWE ciphertexts in base $B = 2^2$. Integer multiplication is then implemented by a digit-wise procedure that follows a schoolbook long multiplication on these radix digits: all small-digit products and carry updates are realized by evaluating lookup tables via programmable bootstrapping (PBS) on LWE ciphertexts, using the generic bi-variate trick, to homomorphically compute the digit product and carry bits. In contrast, our work changes the multiplication algorithmic structure: by lifting blocks to GGSW and using CBS and VP, we reduce the number of bootstraps and LUT evaluations required per n-bit product, so any future low-level PBS optimizations in TFHE libraries translate into further gains for our algorithms.

Table 6. Execution time (s) of ciphertext–plaintext dot product (CxP-DP) for different vector sizes m and thread counts. The input bit size is 16 bits. All executions target $P_{\text{fail}}=2^{-128}$. Running [22] with no bucket doubling and window size 1 for fair comparison.

Size m	**Threads**	**[22] (s)**	**Ours (s)**	**Speedup** ($\times$)
	1	1.43	**0.64**	2.23
2	8	0.23	**0.18**	1.27
	1	2.53	**1.51**	1.67
4	8	0.38	**0.32**	1.18
	1	4.58	**3.26**	1.40
8	8	0.70	**0.61**	1.14
	1	8.66	**7.21**	1.20
16	8	1.21	**1.10**	1.10

As discussed, the current state-of-the-art for TFHE integer scalar multiplication is SMOOTHIE [22] by Pottier et al., which proposes new ciphertext-scalar multiplication and ciphertext-scalar dot-product algorithms directly on top of `tfhe-rs`. They adopt the bucket method known from elliptic-curve multi-scalar multiplication, and adapt it to TFHE and introduce TFHE-specific tricks, such as a negation-with-offset strategy that avoids expensive carry propagation after negation, power-shift aggregation, and bucket-merging techniques that aggressively cut the number of bootstraps, as well as a bucket-doubling scheme that removes per-input precomputation in the multi-scalar case. Our work, however, utilizes a different paradigm, adopting Vertical Packing as our main operation to compute multiplication, by decomposing it to LUTs of a large size $B = 16$ that we found to be optimal. This approach yields measured speedups ranging from 1.10× to 3.46× across different benchmarks, due to our VP paradigm that handles both CxP and CxC effectively.

6 Concluding Remarks

In this work, we propose a novel multiplication algorithm that decomposes the operation into 16-bit Look-Up Tables (LUTs), which are then efficiently computed using Vertical Packing (VP). Our framework introduces new algorithms for both ciphertext-plaintext and ciphertext-ciphertext multiplication, as well as an efficient ciphertext-plaintext dot product. We utilize Circuit Bootstrapping (CBS) to refresh ciphertexts, which ensures composability for the proposed operations. Furthermore, our approach features novel parallelization strategies and utilizes the batched CBS to maximize throughput and efficiency. Our experimental evaluation on selected benchmarks demonstrates significant speedups ranging from 1.10× to 3.46× compared to the baseline `tfhe-rs` implementation and the current state-of-the-art in [22]. Overall, our work establishes a new foundation for high-performance arithmetic in TFHE.

Acknowledgments

This work has been supported by NSF Award #2239334.

References

1. Martin R Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *Cryptology ePrint Archive*, 2015.
2. Loris Bergerat, Anas Boudi, Quentin Bourgerie, Ilaria Chillotti, Damien Ligier, Jean-Baptiste Orfila, and Samuel Tap. Parameter optimization and larger precision for (T)FHE. *Journal of Cryptology*, 36(3):28, 2023.
3. Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (Leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory (TOCT)*, 6(3):1–36, 2014.
4. Sergiu Carpov, Malika Izabachène, and Victor Mollimard. New techniques for multi-value input homomorphic evaluation and applications. In *Cryptographers' Track at the RSA Conference*, pages 106–126. Springer, 2019.
5. Yangyi Chen, Bo Peng, XiaoFeng Wang, and Haixu Tang. Large-scale privacy-preserving mapping of human genomic sequences on hybrid clouds. In *NDSS*, 2012.
6. Jung Hee Cheon, Hyeongmin Choe, Alain Passelègue, Damien Stehlé, and Elias Suvanto. Attacks against the IND-CPAD security of exact FHE schemes. In *ACM Conference on Computer and Communications Security*, pages 2505–2519, 2024.
7. Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *ASIACRYPT*, pages 409–437. Springer, 2017.
8. Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In *ASIACRYPT*, pages 3–33. Springer, 2016.
9. Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: Fast Fully Homomorphic Encryption over the Torus. *Journal of Cryptology*, 33(1):34–91, 2020.
10. Ilaria Chillotti, Damien Ligier, Jean-Baptiste Orfila, and Samuel Tap. Improved programmable bootstrapping with larger precision and efficient arithmetic circuits for TFHE. In *ASIACRYPT*, pages 670–699. Springer, 2021.
11. Joan Daemen and Vincent Rijmen. AES proposal: Rijndael, 1999.
12. Thomas de Ruijter, Jan-Pieter D'Anvers, and Ingrid Verbauwhede. Don't be mean: Reducing approximation noise in TFHE through mean compensation. Cryptology ePrint Archive, Report 2025/809, 2025.
13. Léo Ducas and Daniele Micciancio. FHEW: Bootstrapping Homomorphic Encryption in Less Than a Second. In *EUROCRYPT*, pages 617–640. Springer, 2015.
14. Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Report 2012/144, 2012.
15. Álvaro López García, Jesús Marco De Lucas, Marica Antonacci, Wolfgang Zu Castell, Mario David, Marcus Hardt, Lara Lloret Iglesias, Germán Moltó, Marcin Plociennik, Viet Tran, et al. A cloud-based framework for machine learning workloads and applications. *IEEE access*, 8:18681–18692, 2020.
16. Craig Gentry. Fully homomorphic encryption using ideal lattices. In *ACM Symposium on Theory of Computing*, page 169–178, New York, NY, USA, 2009. Association for Computing Machinery.

17. Tahseen Khan, Wenhong Tian, Guangyao Zhou, Shashikant Ilager, Mingming Gong, and Rajkumar Buyya. Machine learning (ML)-centric resource management in cloud computing: A review and future directions. *Journal of Network and Computer Applications*, 204:103405, 2022.
18. Ben Langmead and Abhinav Nellore. Cloud computing for genomic data analysis and collaboration. *Nature Reviews Genetics*, 19(4):208–219, 2018.
19. Bo Liu, Ravi K Madduri, Borja Sotomayor, Kyle Chard, Lukasz Lacinski, Utpal J Dave, Jianqiang Li, Chunchen Liu, and Ian T Foster. Cloud-based bioinformatics workflow platform for large-scale next-generation sequencing analyses. *Journal of biomedical informatics*, 49:119–133, 2014.
20. Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In *EUROCRYPT*, pages 1–23. Springer, 2010.
21. Luis Montero, Jordan Frery, Celia Kherfallah, Roman Bredehoft, and Andrei Stoian. Machine learning training on encrypted data with tfhe. In *Proceedings of the 10th ACM International Workshop on Security and Privacy Analytics*, pages 71–76, 2024.
22. Xander Pottier, Jan-Pieter D'Anvers, Thomas de Ruijter, and Ingrid Verbauwhede. SMOOTHIE: (Multi-)Scalar Multiplication Optimizations On TFHE. *Cryptology ePrint Archive, Paper 2025/1267*, 2025.
23. Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM*, 56(6):1–40, 2009.
24. Mauro Ribeiro, Katarina Grolinger, and Miriam AM Capretz. Mlaas: Machine learning as a service. In *2015 IEEE 14th international conference on machine learning and applications (ICMLA)*, pages 896–902. IEEE, 2015.
25. Andrei Stoian, Jordan Frery, Roman Bredehoft, Luis Montero, Celia Kherfallah, and Benoit Chevallier-Mames. Deep neural networks for encrypted inference with tfhe. In *International Symposium on Cyber Security, Cryptology, and Machine Learning*, pages 493–500. Springer, 2023.
26. Ruida Wang, Jincheol Ha, Xuan Shen, Xianhui Lu, Chunling Chen, Kunpeng Wang, and Jooyoung Lee. Refined TFHE leveled homomorphic evaluation and its application. Cryptology ePrint Archive, Report 2024/1318, 2024.
27. Ruida Wang, Benqiang Wei, Zhihao Li, Xianhui Lu, and Kunpeng Wang. TFHE Bootstrapping: Faster, Smaller and Time-Space Trade-Offs. In *Australasian Conference on Information Security and Privacy*, pages 196–216. Springer, 2024.
28. Ruida Wang, Yundi Wen, Zhihao Li, Xianhui Lu, Benqiang Wei, Kun Liu, and Kunpeng Wang. Circuit bootstrapping: faster and smaller. In *EUROCRYPT*, pages 342–372. Springer, 2024.
29. Stephen S Yau and Ho G An. Protection of users' data confidentiality in cloud computing. In *Proceedings of the Second Asia-Pacific Symposium on Internetware*, pages 1–6, 2010.
30. Zama. TFHE-rs: A Pure Rust Implementation of the TFHE Scheme for Boolean and Integer Arithmetics Over Encrypted Data, 2022. `https://github.com/zama-ai/tfhe-rs`.
31. Yinqian Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. Cross-VM side channels and their use to extract private keys. In *ACM Computer and Communications Security*, pages 305–316, 2012.
32. Yinqian Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. Cross-tenant side-channel attacks in PaaS clouds. In *ACM Computer and Communications Security*, pages 990–1003, 2014.